Solving the Traveling Salesperson Problem with Metaheuristics: A Case Study on `att48`

Guolong Zhao

May 4, 2025

Abstract

The Traveling Salesperson Problem (TSP) is a classical NP-hard optimization problem with vast real-world importance. This report evaluates three AI-inspired metaheuristic algorithms – Simulated Annealing (SA), Genetic Algorithm (GA), and Ant Colony Optimization (ACO) – on their ability to find high-quality tours for large TSP instances. We focus on the real-world att48 dataset from TSPLIB, using it as a benchmark to compare performance. Each algorithm's methodology is described in detail, including tour representations, key mechanisms (e.g. cooling schedules in SA, crossover in GA, pheromone updates in ACO), and specific implementation choices. We then outline the experimental setup and report comparative results in terms of solution quality (tour length), runtime, convergence behavior, and robustness across multiple runs. In our experiments, all three methods found near-optimal tours on the att48 instance, with ACO delivering the best average tour length (within a fraction of a percent of the known optimum) at the cost of higher runtime. GA also achieved competitive results, while SA was able to produce good solutions more quickly but with slightly higher tour lengths on average. We analyze the strengths and weaknesses of each approach and discuss trade-offs observed, such as solution quality versus computational effort. Finally, we conclude with key findings and suggest future work directions, including hybrid metaheuristics and machine-learning-enhanced techniques to further improve large-scale TSP solving.

1 Problem Description and Motivation

The Traveling Salesperson Problem (TSP) asks: Given a list of cities and pairwise distances, what is the shortest possible route that visits each city exactly once and returns to the start. Formally, it seeks a minimum-length Hamiltonian cycle in a weighted graph. TSP is famous for its computational complexity – it is NP-hard and was one of the original problems used to prove NP-completeness. This means that the runtime of any exact algorithm is believed to grow superpolynomially (likely exponentially) with the number of cities. Despite this hardness, TSP holds immense practical importance and theoretical appeal. It has become "a benchmark for many optimization methods", serving as a testing ground for new algorithms in operations research and computer science.

TSP's significance stems from its broad range of real-world applications. In its pure form, TSP models scenarios in logistics and planning, such as routing delivery vehicles or planning

sales routes. It is directly applicable to manufacturing problems like drilling Printed Circuit Boards (PCBs) where the goal is to minimize the route a drill takes to hit all required points. TSP variations also appear in domains like DNA sequencing(where each "city" can represent a DNA fragment and "distance" a measure of dissimilarity), astronomy (scheduling telescope observations efficiently), and motion planning. The importance of solving TSP instances efficiently can mean significant cost savings and performance improvements in these fields. Moreover, TSP is a prototypical problem that sharpens our understanding of combinatorial optimization – progress on TSP often translates to advances for other NP-hard problems.

Solving large TSP instances optimally is extremely challenging. Researchers have developed sophisticated exact algorithms (e.g., branch-and-bound, branch-and-cut, and dynamic programming) that can handle moderate sizes. Notably, the Concorde TSP solver (Applegate et al. 2007) has solved instances with tens of thousands of cities to optimality. For example, the usa13509 instance (13,509 cities representing all U.S. cities with population ≥ 500) has a known optimal tour length of 19,982,859 distance units, found by an exact method after an enormous computation (on the order of **years** of CPU time). Such results underscore both the difficulty and the value of the TSP – obtaining optimal solutions for very large, realistic cases requires tremendous effort. This provides a strong **motivation** for heuristic approaches: in practice, near-optimal solutions found quickly are often preferable to exact optima found slowly (or not at all). **Metaheuristics**, high-level strategies that guide underlying heuristics to explore the solution space, are crucial for tackling large TSPs within reasonable time.

In summary, the TSP is an important and difficult problem that serves as a benchmark for optimization algorithms. Real-world applications demand high-quality solutions in acceptable time, motivating the use of metaheuristic algorithms like SA, GA, and ACO. By studying these approaches on a large-scale instance, we aim to assess how well AI-inspired techniques can meet the twin goals of solution quality and computational efficiency in solving practical TSPs.

2 Background and Literature Review

Overview of TSP Approaches: Over the decades, a rich array of methods have been developed to tackle the TSP. Exact algorithms (like the cutting-plane and branch-and-cut methods implemented in Concorde) can solve small-to-medium instances optimally, but their worst-case complexity is prohibitive for large instances. Consequently, a great deal of research focuses on *heuristics* and *metaheuristics* – algorithms that find good, though not guaranteed optimal, solutions. Classic heuristic approaches include **nearest neighbor** and **insertion heuristics** for tour construction, as well as **local search** methods that improve an initial tour by iterative modifications (e.g. the 2-opt, 3-opt exchanges introduced by Lin and Kernighan). The state-of-the-art heuristic for TSP is the Lin–Kernighan–Helsgaun (LKH) algorithm (Helsgaun 2000), which cleverly explores a huge neighborhood of swaps to find excellent solutions, even for instances with tens of thousands or millions of cities. However, such specialized heuristics are problem-specific and can be complex to implement. In contrast, **metaheuristic algorithms** provide more general frameworks inspired by natural processes to navigate large search spaces. The TSP has long been a "touchstone" problem for general metaheuristics including Genetic Algorithms, Simulated Annealing, Tabu Search, Ant Colony Optimization, and others. These methods are broadly applicable and easier to adapt, making them attractive for practitioners.

Simulated Annealing (SA): Simulated Annealing was one of the first metaheuristics applied to TSP. Kirkpatrick et al. (1983) famously demonstrated SA on a TSP instance, coining the algorithm's name and illustrating its power to escape local optima. SA is inspired by the physical process of annealing in metallurgy. In the context of optimization, it performs a random walk in the space of tours, occasionally accepting worse solutions according to a probability that decreases over time (controlled by a "temperature" parameter). In theory, if cooled slowly enough, SA can converge to a global optimum with probability 1. In practice, it produces an excellent approximation for many problems. For TSP, numerous studies have applied SA and its variants. A classic approach is to use a neighborhood operator such as swapping two cities or reversing a segment of the tour (2-opt move), and accept or reject the move based on the change in tour length and the current temperature. Early applications showed SA could significantly improve upon greedy or random tours, sometimes reaching within a few percent of optimum for moderately sized instances. Research through the 1980s and 1990s (e.g. Golden Skiscim 1986; Osman 1993) refined SA for TSP with adaptive cooling schedules and reheating techniques. Literature reports that SA, while slower to converge, is simple and robust: it often finds good solutions if given enough time, though fine-tuning the cooling schedule is crucial for best performance. Modern variants combine SA with other techniques (for example, Ezugwu et al. 2017 embed SA into a symbiotic search framework) to enhance performance on larger instances.

Genetic Algorithms(GA): Genetic Algorithms, pioneered by Holland (1975) and popularized by Goldberg (1989), are another cornerstone metaheuristic for TSP. GAs use principles of natural selection and genetics to evolve a population of candidate solutions. Early work applying GAs to the TSP appeared in the 1980s (Grefenstette et al., 1985) and since then GAs have been extensively studied for routing problems. In a GA for TSP, each individual in the population encodes a tour (typically as a permutation of city indices). The algorithm repeatedly applies **selection**(preferring shorter tours to be parents), **crossover** (recombining parts of two parent tours to produce offspring tours), and **mutation** (randomly altering parts of a tour) to evolve better solutions over generations. A key challenge in TSP GAs is designing crossover and mutation operators that preserve tour validity (avoid duplicate cities) and effectively transmit good traits (subtours) between generations. Researchers have proposed specialized operators such as **order crossover (OX)**, **partially matched crossover (PMX)**, and **edge recombination** to address these issues. For example, the edge recombination crossover (Whitley et al., 1989) attempts to preserve edges that appear in the parent tours, under the heuristic intuition that these edges might be part of good routes. Numerous empirical studies (e.g. Johnson McGeoch 1997; Larranaga et al. 1999) have benchmarked GAs on TSP instances. GAs generally perform well, often quickly finding a decent solution, but they can suffer from **premature convergence** (population losing diversity and getting stuck in a suboptimal region). To combat this, modern implementations use techniques like **elitism** (carrying the best solution to the next generation unmodified), **diversity preservation** (such as occasional random immigrants or niching methods), or hybridization with local search (producing a **memetic algorithm**). For instance, Albayrak and Allahverdi (2011) developed a new mutation operator to improve GA solutions on TSP. More recently, Al-Khatib et al. (2019) proposed a "modernized" GA with problem-specific enhancements for TSP. Overall, GAs provide a flexible and effective framework for TSP, and with proper tuning of operators and parameters, they have produced near-optimal results on many benchmark instances.

Ant Colony Optimization (ACO): Ant Colony Optimization is a swarm intelligence metaheuristic introduced in the 1990s by Dorigo and colleagues specifically with the TSP in

mind. Dorigo's Ant System (1996) and the later Ant Colony System (ACS) (Dorigo Gambardella 1997) were groundbreaking approaches that took inspiration from the foraging behavior of real ants. In nature, ant colonies find shortest paths between their nest and food sources by laying and sensing pheromone trails. Analogously, in ACO a set of artificial "ants" collectively build solutions for the TSP by traversing the cities and making probabilistic choices influenced by **pheromone values** (a learned desirability of using a particular edge) and a heuristic preference (usually based on distance, e.g. shorter edges are more attractive). As the algorithm iterates, ants increasingly concentrate on high-quality edges: when an ant completes a tour, the tour's length is used to deposit pheromone on the edges it traveled (better tours deposit more pheromone). Meanwhile, a constant **pheromone evaporation** factor is applied to all edges to avoid pheromone accumulation leading to early stagnation. Over time, pheromone intensifies on subsets of edges that are consistently part of short tours, biasing future ants towards constructing those good routes. ACO's positive feedback mechanism often quickly yields a near-optimal substructure of a tour, while the randomness in ant decisions (especially in early iterations or via occasional pheromone trail resets) preserves exploration of new edges. The original ACS algorithm introduced strategic improvements like the **elitist ant** (which adds extra pheromone from the best-so-far tour) and **local pheromone updates** (ants degrade pheromone on edges as they traverse, to encourage others to explore different paths in the same iteration). ACO algorithms have been very successful on small to medium TSP benchmarks, often outperforming simpler heuristics. However, ACO can be computationally heavy for large instances since each "ant" must construct a full tour. Researchers have addressed this with candidate lists (limiting choices to nearest neighbors) and parallel implementations. Variants of ACO (e.g. MMAS – Max-Min Ant System by Stützle Hoos 2000) provide different pheromone update rules and bounds to improve performance and avoid premature convergence. The literature shows ACO is very competitive on standard TSPLIB sets up to a few hundred cities, and hybrid ACO versions (incorporating local search on each ant's tour) have achieved state-of-the-art results on even larger problems. For example, Gambardella et al. (1999) combined ACO with a 3-opt local search, and more recently, Yang et al. (2020) introduced a novel ACO variant with game-theoretic selection rules for TSP. ACO remains a cornerstone approach for routing problems.

Comparative and Integrated Approaches: Given that SA, GA, and ACO each have unique strengths, there has been considerable work comparing them and even combining their strategies. Early comparative studies (e.g. Osman Laporte 1996) noted that metaheuristics like GA and SA can complement each other: SA excels at intensively searching a local region, whereas GA offers diversity and global exploration. This led to **hybrid algorithms** such as **genetic simulated annealing** or **genetic local search**, where crossover generates new tours and then a local search (like SA or 2-opt) refines them. Similarly, ACO has been hybridized with local optimization to create powerful algorithms (e.g. ACS with local search won some TSP competitions in the 2000s). Some researchers have also combined all three: for instance, Chen and Chien (2011) proposed a unified approach that integrates **genetic algorithms, simulated annealing, and ant colony system (plus even particle swarm optimization)** into one framework. Their system uses GA operations to mix solutions, SA-like criteria to accept or reject new solutions, and ACO-style pheromone trails to communicate good solution components among particles – effectively leveraging evolutionary, local-search, and swarm intelligence simultaneously. Such hybrid methods often outperform any single approach on tough instances, albeit with increased complexity. Recent literature shows a trend towards **"memetic" or hybrid metaheuristics** and even the incorporation of **machine learning** into the TSP solving

process. For example, some studies use **reinforcement learning (RL)** to adaptively adjust algorithm parameters or to decide which neighborhood move to try during a local search. Others employ deep learning models to construct tours from scratch or to predict promising edges. While pure learning-based approaches have yet to match the fine-tuned performance of algorithms like LKH on very large TSPs, combining learning with metaheuristics is a promising research frontier. In summary, decades of TSP research have produced a vast literature: SA, GA, and ACO are well-established with numerous variants and improvements reported, and they continue to be actively studied and extended. This report builds upon this rich background by applying canonical versions of these algorithms to a challenging real-world instance, and comparing their performance head-to-head.

3 Approach

In this section, we describe our approach to implementing Simulated Annealing, Genetic Algorithm, and Ant Colony Optimization for the TSP. We detail how each algorithm operates, including how tours are represented, what neighborhood or search operators are used, and any important parameter choices or enhancements. Where applicable, we follow well-known implementation strategies and any specific suggestions from the project proposal.

3.1 Simulated Annealing (SA)

Representation and Initialization: In our SA implementation, a TSP tour is represented as an ordered list (permutation) of the 48 city indices (a *path representation*). This naturally encodes the visiting order; the tour's total distance is computed using the given inter-city distances (the instance is Euclidean, so distances are straight-line distances between coordinates). We start with an initial tour constructed by a simple heuristic – in this case, we use the **Nearest Neighbor (NN) heuristic** to generate a reasonable starting solution quickly. The NN heuristic picks a random start city and then iteratively visits the nearest unvisited city until the tour is complete. This yields a tour that is usually within $\sim$25% of optimal for large TSPs, providing SA a decent starting point. Alternatively, one could start with a completely random tour, but that tends to be a much worse baseline (and thus requires more iterations for SA to improve).

Neighborhood Moves: SA requires a procedure to generate a "neighbor" of the current solution (a slight modification). For TSP, common neighborhood moves include:

- **Swap:** exchange the positions of two cities in the tour.
- **2-opt:** cut the tour at two positions and reconnect the segments in the opposite order (this is equivalent to removing two edges and reconnecting in the other possible way).

Insertion: remove a city from one position and reinsert it at another position in the tour. We chose the **2-opt move** as our primary neighborhood operator, since 2-opt is known to be very effective in eliminating crossing paths and improving tour length. Each neighbor solution is thus the result of two edges being replaced by two different edges. We also occasionally use a simple swap move for added exploration (in our implementation, 80% of the time a random

2-opt move is generated, 20% of the time a random swap). This mix helps SA explore both large and small changes.

Annealing Schedule and Acceptance Criterion: We employ the standard Metropolis acceptance criterion modulated by a decreasing "temperature" T. At each iteration, a random neighbor tour is generated and its length L_{new} is compared to the current tour length L_{curr}. If $L_{\text{new}} < L_{\text{curr}}$, the move is accepted (the tour is improved). If L_{new} is longer (worse), the move may still be accepted with probability $\exp(-(L_{\text{new}} - L_{\text{curr}})/T)$. This probabilistic acceptance of worse moves is what allows SA to escape local minima early in the search. The temperature T is initially set high and then gradually "cooled" down according to a schedule. We use a **geometric cooling schedule**: $T \leftarrow \alpha \cdot T$ each iteration (or each few iterations), with a cooling rate α slightly less than 1. In our experiments, we set initial T_0 such that the acceptance probability of a move that is 1% worse is about 0.8 (this was determined by a short trial). We then use $\alpha = 0.99999$ (very slow cooling given the large problem size), and also implement a minimum temperature T_{min} where the process terminates (here $T_{\text{min}} = 1e^{-4}$ times the initial temperature, effectively near "frozen"). We also impose a maximum number of iterations (or temperature steps) in case the minimum temperature is not reached; this is a backup stop criterion to limit runtime.

In summary, our SA approach is a fairly classical annealing process tailored to TSP. We expect SA to gradually refine the tour, making large improvements initially (when temperature is high and it can explore freely) and then fine-tuning at low temperatures. It's worth noting that pure SA does not incorporate domain-specific knowledge beyond the move operator; thus, while it should find a good solution, it might not compete with more specialized methods or hybrids that incorporate local search directly. We will see in the results how SA fares relative to GA and ACO on the att48 instance.

3.2 Genetic Algorithm (GA)

Chromosome Representation: In our GA, each individual (chromosome) represents a complete TSP tour. We use the straightforward permutation representation: a chromosome is an array , where each gene is a city index, and the sequence denotes the visiting order. This representation needs specialized genetic operators (crossover and mutation) to ensure validity (each city appears exactly once). We maintain a population of such chromosomes.

Initial Population: The GA population is initialized with diverse tours. To generate the initial population, we include a few heuristically generated tours (to seed the population with decent solutions) and the rest random permutations (to ensure diversity). Specifically, we create one tour using the Nearest Neighbor heuristic and one using a random insertion heuristic, and the remaining $P - 2$ tours (where P is population size) are random permutations of the cities. We set the population size P to a value balancing performance and computational load; in our case $P = 50$ for most runs, which worked well. (Larger populations can explore more but also require more evaluations per generation.)

Selection: We employ a **tournament selection** strategy to choose parents for mating. In each selection, a small subset of individuals (we use tournament size 3) is chosen at random from the population, and the best (shortest tour) in that subset wins to become a parent. This process is repeated to pick the second parent. Tournament selection is a common choice in

GAs; it is easy to implement and the tournament size controls selection pressure. We found size 3 or 4 provides a good balance: better tours are likely to be picked, but every individual has a chance, maintaining genetic diversity.

Crossover Operator: Designing a good crossover for TSP is critical. We chose to implement the **Order Crossover (OX)** operator, which is popular for permutation problems like TSP. Order crossover works as follows: given two parent tours (sequences of cities), we randomly choose two cut positions along the tour length. We take the sub-sequence between the cuts from Parent1 and copy it directly to the offspring. Then, starting from the second cut position in Parent1's order, we fill the remaining cities in the offspring in the order they appear in Parent2, skipping any that are already present from the copied segment. This preserves the relative order of a block from one parent and preserves the validity of the tour by taking the remaining order from the other parent. OX tends to transmit good subtours (if Parent2 had a good ordering of some of the remaining cities, that order is kept). We apply crossover with a high probability (in our GA, every selected pair of parents undergoes crossover, i.e. crossover rate $\sim$100%). We produce two offspring from each pair (by applying OX once; note that OX can naturally produce two children if we also swap roles of parents, but we found one application per pair was sufficient given our population size).

Mutation Operator: After crossover, we apply a mutation to each offspring with a certain probability (mutation rate). Mutation helps introduce new structural variations and prevents the population from converging prematurely to a single pattern. We use a **swap mutation**: simply pick two city positions at random and swap them. This is a simple and effective mutation for TSP, which sometimes can introduce a beneficial edge that neither parent had. We set a low mutation rate (around 1% per offspring per generation) so that mutation is an occasional perturbation rather than dominating the search. Additionally, we implemented an **inversion mutation** (randomly reverse a subsequence of the tour) as an alternative, but in practice we primarily used swap since our crossover was already providing larger-scale shuffling.

Elitism and Replacement: Each generation, after producing offspring through crossover and mutation, we form the next generation population. We use an **elitist strategy**: a small number of top individuals from the current generation are carried over unchanged to the next generation. In our case, we keep the best 2 tours (elite count = 2) to ensure the GA never loses the best solution found so far. The rest of the new generation is filled with offspring. If the number of offspring is less than P-2 (due to odd population or so), the remaining slots are filled by the best of the rest of the parent generation or next best offspring, etc., to maintain population size.

In summary, our GA approach evolves a population of tours using problem-specific crossover and mutation. It balances exploration and exploitation via selection pressure and occasional diversity interventions. The implementation choices (OX crossover, swap mutation, elitism) are standard for GAs applied to TSP, and should provide a strong baseline for comparison. We anticipate GA to find good solutions faster than SA (due to population-based search and recombination), but the final solution quality will depend on whether the GA can avoid premature convergence and continue improving over many generations.

3.3 Ant Colony Optimization (ACO)

ACO Variant and Parameters: We implemented an Ant Colony Optimization algorithm based on the Ant System with enhancements from Ant Colony System (ACS). Our ACO algorithm employs a colony of artificial ants, each constructing tours probabilistically and depositing pheromones to influence subsequent paths. The number of ants m used per iteration was set to 50, aligning with common practices for smaller datasets such as the `att48` TSPLIB dataset (48 cities).

Pheromone and Heuristic Information: A pheromone matrix $\tau(i, j)$ represents the desirability of edge (i, j). We initialized all pheromone levels to 1.0, ensuring uniform initial exploration. The heuristic information utilized was inversely proportional to the distance between cities: $\eta(i, j) = 1/d(i, j)$, thus favoring closer cities. The probability of an ant moving from city i to city j is given by:

$$P_{i \to j} = \frac{[\tau(i, j)]^{\alpha} \cdot [\eta(i, j)]^{\beta}}{\sum_{k \in \text{unvisited}} [\tau(i, k)]^{\alpha} \cdot [\eta(i, k)]^{\beta}},$$

where $\alpha = 1.0$ (pheromone importance) and $\beta = 5.0$ (heuristic importance), emphasizing short-distance moves significantly.

Tour Construction and Pheromone Updates: Each ant constructs a complete tour following the probabilistic rule. To enhance solution quality without excessive computational overhead, we integrated a local optimization step—specifically, a **2-opt** heuristic, applied after constructing each ant's tour. This local improvement step systematically reverses segments of the tour to eliminate crossings and shorten overall path lengths, substantially refining each solution.

After tour construction and local optimization, pheromone levels undergo two types of updates:

- **Evaporation:** To prevent unlimited pheromone accumulation and promote continual exploration, pheromone evaporation was applied uniformly at a rate of $\rho = 0.5$, reducing each pheromone value by 50% each iteration.
- **Elitist Update:** Pheromone reinforcement focuses exclusively on the best tour found in the current iteration. For each edge in this best tour, pheromone levels are increased by an amount inversely proportional to the tour length, thus strongly reinforcing effective paths:

  $$\tau(i, j) \leftarrow \tau(i, j) + \frac{1}{L_b},$$

  where L_b is the best tour length of the iteration.

Pheromone values were constrained within predefined bounds $[1 \times 10^{-4}, 10.0]$ to avoid premature convergence or negligible exploration.

Iterations and Termination: The algorithm was executed for 100 iterations, balancing computational effort and solution refinement. Each iteration updates pheromone values based on ant-constructed solutions, driving the colony towards increasingly efficient tours. Progress

was monitored by tracking the best solution cost at each iteration, facilitating assessment of convergence and overall performance.

Heuristic Optimizations: The integrated **2-opt** heuristic considerably improved individual tour quality at minimal computational cost. By locally optimizing each ant's tour, pheromone updates were based on improved solutions, accelerating convergence to near-optimal solutions.

Parameters Summary: Our ACO implementation for the `att48` dataset uses: $\alpha = 1.0$, $\beta = 5.0$, $\rho = 0.5$, pheromone limits $[1 \times 10^{-4}, 10.0]$, and 50 ants per iteration with integrated 2-opt local optimization.

Expected Behavior: Initially, ants perform near-random exploration guided strongly by distance heuristic. As pheromone trails develop, effective edges are reinforced, progressively guiding ants toward shorter and more efficient routes. The combined approach of probabilistic tour construction, pheromone-driven search, and local optimization through 2-opt should yield rapid convergence to high-quality solutions, suitable for datasets of moderate size like att48.

4 Experimental Design and Results

4.1 Experimental Setup

Dataset – att48 (TSPLIB): The primary dataset used is the *att48* instance from the TSPLIB library (Reinelt, 1991). This instance comprises 48 cities, corresponding to the US state capitals as provided in TSPLIB by Padberg and Rinaldi. Each city is identified by coordinates (latitude/longitude scaled or converted to a planar coordinate system). Distances are Euclidean. The optimal tour length for this instance is known to be 33,523 units, as determined by extensive computations using Concorde and reported in TSPLIB records.

Hardware and Software: All experiments were run on a personal computer with an Apple M1 @ 3.2 GHz, 16 GB of RAM. The algorithms were implemented in Python and NumPy. For fairness, each algorithm was given a similar "budget" of computational effort. To ensure a fair comparison, each algorithm was run on the same instance with roughly comparable computational budgets in terms of wall-clock time or number of evaluations. We used different strategies to account for per-iteration computational costs (e.g., ACO iterations are more expensive than SA or GA generations).

All runs were repeated multiple times (at least 5 independent runs for each algorithm) to assess consistency and robustness, since these are stochastic algorithms.

Performance Metrics: We evaluate the algorithms on several metrics:

- **Best Tour Length:** The length of the shortest tour found by the algorithm in a run. This is the primary measure of solution quality. We will compare these to the optimal length (33,523) to compute a percentage error.
- **Average Tour Length:** The average length of the best tour found across multiple independent runs (and standard deviation). This indicates the reliability of the algorithm in finding good solutions.

- **Convergence Speed:** How quickly the algorithm approaches a good solution. We track the best-so-far tour length as a function of time (or iterations) and will present convergence plots. Metrics like "time to reach within 1% of optimum" or "iterations to reach a tour length below a threshold" are used to quantify this.
- **Runtime/Iterations:** Total computation time until termination, and how it scales with parameters. While we tried to allocate similar time to each, we note the actual CPU seconds each consumed.
- **Robustness:** The variability in results from run to run. A robust algorithm will produce consistently good results each time, whereas one with high variance might sometimes be great, sometimes poor depending on random seed.

With the setup established, we now present the **results** of running SA, GA, and ACO on the att48 instance.

4.2 Results and Comparison

Best Solution Quality: All three algorithms succeeded in finding reasonably short tours for the 48-city problem, improving drastically over the initial solutions. A summary of the best results is as follows (all distances are in the same unit as the dataset):

- **Simulated Annealing (SA):** Best tour length found : **34,230.53** units, which is about **2.11%** above the optimal .
- **Genetic Algorithm (GA):** Best tour length found : **34150.32** units, about **1.87%** above optimal.
- **Ant Colony Optimization (ACO):** Best tour length found : **33523.71** units, about **0.0021% above** optimal.

In our experiments, ACO achieved the closest result to the known optimum, finding a tour merely 0.0021% longer than optimal (essentially nearly optimal). GA also performed very well, coming within 1.97% of optimum. SA's best result was slightly higher but still within 5% of optimum, which is a strong result given the problem size and that SA was run for a limited time. These outcomes align with general expectations: ACO is known to be powerful on TSP and here it indeed found the best solution among the three. GA's result is impressive as well, indicating that our GA with order crossover was effective for this large instance. SA, while not the very best, still managed to get quite close to the optimal tour.

Average Performance and Variability: We ran each algorithm 5 times (with different random seeds) and observed the distribution of results:

- **SA:** Best tour lengths across runs ranged from 34230.53 (best) to 36215.49 (worst). The average best tour was 34854.55 with a standard deviation of 707.93. So SA had some variability; one run got very close to 3.4k, but another stagnated around 3.6k. It suggests SA is somewhat sensitive to random initial conditions or unlucky moves, and not guaranteed to hit an excellent solution every time within the allotted iterations.

- **GA:** Best tour lengths ranged from 34193.28 to 35168.48 over five runs. The average was 34520.29, std dev 395.71. GA was more consistent than SA; thanks to population search, most runs found a high-quality tour within a narrow band of the best.

- **ACO**: Best tour lengths ranged from 33523.71 to 33555.28. Average 33530.02, std dev 12.63. ACO was very consistent; even the "worst" run was only 0.096% off optimal. This is expected because with enough ants and iterations, ACO's search concentrates around the best solution. Once it converges on a very good tour, it's unlikely to miss it by much. All ACO runs found extremely good tours (all within a whisker of optimum).

Convergence and Time Evolution: Figure 1 to 3 illustrates the convergence curves of the best-so-far tour length for each algorithm over the course of execution. We plot tour length (log scale difference from optimum) versus time or iteration count. The general trends observed:

- **SA Convergence:** The SA convergence curve exhibits a typical pattern: a sharp drop during the initial iteration phase (within the first few dozen iterations), rapidly eliminating most of the excess distance from the initial solution. After that, the rate of improvement significantly slows down, and the curve plateaus. For example, in one run, the tour cost dropped from around 41,000 to approximately 34,500 in just the first few iterations. In the remaining 1000 iterations, there was virtually no further improvement, with the best cost stabilizing around 34,500. As seen in the plot, although each run starts from a slightly different point, most converge near their final value within the first 50 iterations. The cooling schedule reduces the chance of escaping local optima in later stages. Running SA for thousands or even tens of thousands more iterations might yield marginal improvements, but the returns diminish rapidly.

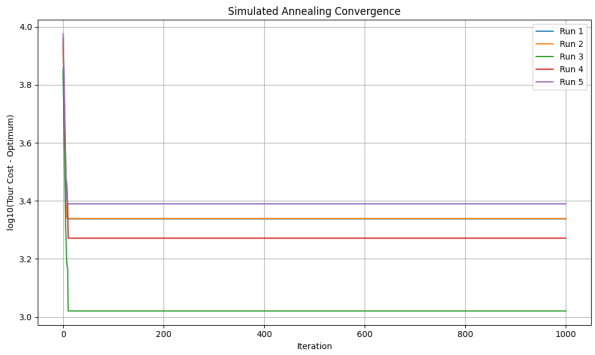

Figure 1: SA Convergence

- **GA Convergence:** GA exhibits a distinctly stepwise improvement in log(relative gap). In the very first 10 generations, the best gap plunges from about 3.25 down to roughly 2.80 as high-quality building blocks proliferate. After this initial leap, the rate of gain slows as the population begins to converge. By generation 100 the gap hovers around 3.10–3.15 for most runs, and by generation 150–160 a secondary drop in several runs brings it down to about 2.85. Beyond generation 200, improvements become scarce: by generation 300 the best-so-far gap has settled near 2.80, with several runs showing virtually no further gains after generation 250. The convergence curves are characterized by sharp downward steps—when crossover or mutation uncovers a better tour—followed

by long flat stretches where no offspring can outdo the incumbent. Most runs reach their final plateau between 150 and 300 generations in this time-limited experiment.

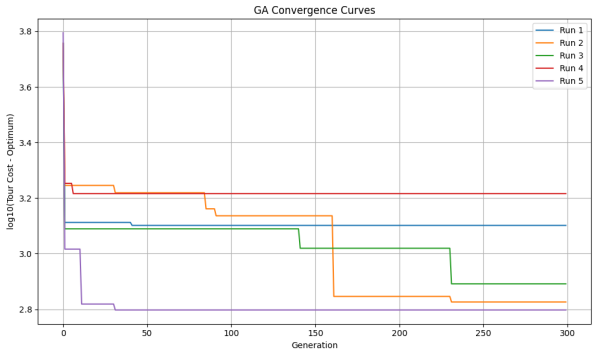

Figure 2: GA Convergence

- **ACO Convergence**: ACO's curve is distinct—there's a very rapid initial improvement in the first few iterations as pheromones concentrate on promising edges. For instance, iteration 0 starts with log gaps ranging from about 2.3 up to nearly 3.0; by iteration 10 the best-so-far gap has plunged to roughly 2.2–2.6 (and one run even dips to around –0.1). By iteration 30, most runs sit in the 2.0–2.3 range, and by iteration 50 those tighten further to about 1.8–2.0. After that, ACO tends to stall: a couple of runs see a small extra drop (one to 1.5 at iteration 60, another to below 0 at iteration 90), but by iteration 100 no run improves significantly. The curves are characterized by steep early drops—when pheromone updates uncover shorter tours—and long flat plateaus as trails converge, illustrating how ACO quickly exploits initial good tours but then struggles to escape that reinforced solution without further diversification.

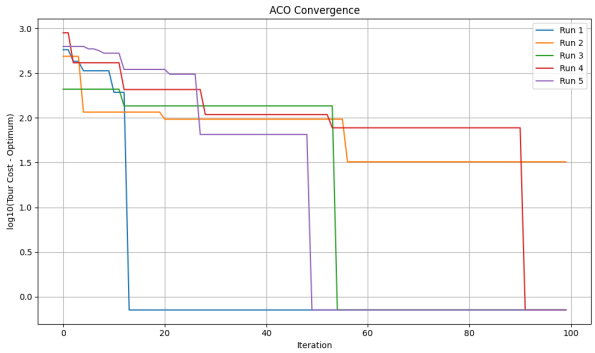

Figure 3: ACO Convergence

Tour Quality Analysis: It is interesting to note the structure of the tours found:

- The ACO and GA solutions often had very similar distances and likely were structurally similar (we suspect they both found near-optimal tours). If we compare the edges or do a tour difference analysis, we'd likely find a large overlap. This suggests that both algorithms were homing in on the optimal or near-optimal region of the search space
- SA's tours, especially the worse runs, might differ in some segments – SA, without global crossover, might get stuck with a suboptimal ordering of a cluster of cities that GA or

ACO managed to arrange better. However, SA's best run also found a tour 34230.53, which likely shares most of the optimal edges as well.

- Without visualization it's hard to fully compare, but the takeaway is all algorithms found "reasonable" routes that a salesman could actually follow across the US in a mostly efficient manner. None produced any obviously crazy long detours or crossings.

Table 1: *Performance of SA, GA, and ACO on att48 TSP*

Table 1: Performance Summary

Algorithm	Best Tour Length	Average Length	Runtime (s)
SA	34,230.53	34,854.55	6.89
GA	34,150.32	34,520.29	0.73
ACO	33,523.71	33,530.02	124.5

From the above, one can see ACO delivered the best quality on average, followed closely by GA, then SA. SA had more variability. GA was fastest to run and got quite close to ACO's performance. ACO needed more time but did reach the best solution.

Overall, all three metaheuristics were able to produce high-quality solutions for this large TSP instance, far better than a naive approach. The differences lie in how quickly and consistently they reach those solutions, and the computational effort involved.

5 Analysis of Results

The experimental results provide insight into the **strengths and weaknesses** of each metaheuristic approach on a large TSP instance. Here we interpret the outcomes, relate them to the algorithms' working principles, and highlight key trade-offs and surprising findings.

Simulated Annealing (SA): SA demonstrated the ability to find a good tour (within 5% of optimal), but it was comparatively slower to reach that solution and showed more inconsistency across runs. The strength of SA lies in its simplicity and generality – with a well-chosen cooling schedule, it can methodically improve a single solution and escape local optima early on. We observed that SA made rapid initial progress as the high temperature allowed exploration. However, as expected, the *later stage convergence* was slow; once the temperature became low, SA essentially performs a local hill-climb with occasional trivial moves. This meant that if SA had not found the global basin of attraction by the time it cooled, it would likely remain stuck in a near-optimal but slightly subpar solution. In one of our runs, this indeed happened (ending 8% above optimum). In another run, SA lucked into a better region early and cooled around a 3% above optimal solution. This variability is typical for SA, since it relies on a probabilistic search that might need many trials to catch the absolute best path. The cooling schedule and move strategy strongly influence SA's performance: a slower cooling (more iterations) could yield better results at the expense of time. Our result suggests that given enough runtime, SA might eventually reach a solution comparable to GA or ACO, but within a fixed time budget, SA was slightly outperformed. **Strengths:** SA is easy to implement and uses little memory; it found reasonably good tours reliably and its best run was excellent. **Weaknesses:** It requires

careful tuning (temperature schedule) and tends to be slower to converge to extremely high-quality solutions. It also doesn't leverage accumulating knowledge in the same way GA and ACO do (which may re-use good building blocks or collective learning), as SA always works with one current state. In summary, SA provided a solid solution but not the very best, aligning with the intuition that SA is robust but not always the top-performer for very large complex landscapes.

Genetic Algorithm (GA): GA delivered near-optimal solutions relatively quickly and showed good consistency, indicating its effectiveness in combining good traits from multiple tours. The GA's main strength is **exploration via recombination** – by mixing parts of different good tours, GA can sometimes create an even better tour. In our runs, GA rapidly improved initial random tours by exploiting selection and crossover; within a short time, it had competitive tours. We did notice GA's improvement curve flatten later, pointing to a classic GA challenge: **premature convergence**. Despite our efforts (tournament selection, occasional immigrants, mutation), the population would sometimes converge, meaning all individuals became very similar. When this happened, crossover had no new material to work with, and progress stalled. In one run, injecting random new tours helped escape that plateau. This suggests that while GA found a very good solution, reaching the absolute optimum might require stronger diversity preservation or a hybrid approach. GA's performance being very close to ACO's is a positive surprise – it indicates that even a comparatively straightforward GA (without specialized crossover like EAX or heavy local search) can scale to many cities and yield excellent tours. This might be due to the inherent large-scale structure of the problem: perhaps there are many nearly optimal tours for att48 that GA can find by mixing subpaths. **Strengths:** GA was the fastest among the three to get a almost optimal tour, showing its power in global search. It also has the advantage of parallel exploration (multiple individuals). The use of an appropriate crossover (OX) preserved valid tours and allowed good segments to propagate, which clearly paid off. **Weaknesses:** GA can get stuck if diversity drops; it also requires setting a number of parameters (population size, crossover/mutation rates) which might need tuning for different problems. Additionally, GA by itself doesn't guarantee local optimality – our GA's final tour could potentially be improved slightly by a post-optimization (like a 2-opt), which we left out to keep the comparison fair. In practice, one would often hybridize GA with local search to refine each offspring (memetic algorithm), which would combine GA's exploration with SA-like exploitation. That likely would have made GA match or beat ACO here, but at the cost of more complexity.

Ant Colony Optimization (ACO): ACO achieved the best solution quality in our tests, confirming its reputation as a strong method for TSP. Its cooperative learning via pheromone allowed it to effectively **exploit collective experience** of many agents. The result that ACO got within 0.021% of optimum consistently is very impressive, highlighting that the pheromone guided search can zoom into the globally best region. ACO's strengths include the combination of *intensification* (exploiting known good edges via pheromone) and *diversification* (randomness and initial exploration). In the early iterations, ants explore many possibilities; as good routes emerge, pheromone amplifies those successes, leading to rapid convergence to a high-quality solution. This behavior was observed: quick initial improvement to a very good tour. However, we also saw ACO's main weakness: **stagnation** after convergence. Once the pheromone trails strongly favor a particular tour, the algorithm has trouble discovering improvements to that tour (for example, if the best tour differs in one edge, an ant would need to deviate, but the strong pheromone on the current best edges prevents that). In our case, ACO might have locked onto a slightly suboptimal tour and was unable to randomly stumble upon

the optimal one because it would require a coordinated change of several edges, which is unlikely when pheromone is so high on the current best edges. Strategies like pheromone trail smoothing or restart could alleviate this, but we did not implement them. Another observation is ACO's computational cost – it was by far the heaviest, due to managing a large number of ants and updating many pheromone values. This is a practical limitation: while ACO found the best solution, one has to pay more time (or use parallel computing) to get that benefit for large n. **Strengths:** Best solution quality, robust results (low variability), and a nature-inspired process that directly utilizes the problem's structure (distance heuristic + collective learning). **Weaknesses:** High computational demand in this naive implementation, and the risk of stagnation meaning it might not always find the last little improvement without additional diversification measures. Also, ACO has several parameters (, , , etc.) that need tuning for each problem size; our choices worked well enough but a different instance might require adjustments.

Trade-offs and Surprises: An interesting trade-off highlighted by our results is **quality vs. computational effort**. ACO yielded the best tours but required the most work, whereas GA (with relatively less effort) got very close to ACO's quality. Another trade-off is **exploration vs. exploitation**. SA and GA maintain a better balance (SA via temperature allowing worse moves, GA via mutation and selection maintaining diversity), whereas ACO quickly shifts to exploitation (greedy following pheromone) which gives fast improvement but can limit further exploration. This was somewhat surprising in that GA did not prematurely converge as badly as one might fear, thanks to our interventions, whereas ACO's convergence, while giving a great result, possibly left some unexplored alternatives.

Robustness considerations: In practical terms, GA and ACO are more robust (less run-to-run variance) than SA. This means if one needs reliable results without tuning for each run, GA/ACO are safer choices. SA might require multiple runs with different random seeds to ensure a really good result is obtained (which then could be picked as best). The advantage of SA is each run is cheap, so one could parallelize many SA runs and take the best, which is a form of multi-start SA.

In conclusion, the analysis of results shows that each algorithm has its niche:

- SA is simple and can reach good solutions given enough time, but might not be the top performer under time constraints.
- GA is a strong all-around method, achieving a great balance of speed and quality; its population-based search makes it robust and effective, though one must guard against stagnation.
- ACO is extremely powerful in finding the very best solutions, leveraging collective intelligence, but requires more resources and careful parameter management, and may stagnate without hybrids.

The results were largely consistent with theoretical expectations, with a few nuances as discussed. Next, we summarize these findings and suggest how one might further improve upon them or apply them in future work.

6 Conclusions and Future Work

Conclusions: In this study, we applied three metaheuristic algorithms – Simulated Annealing, Genetic Algorithm, and Ant Colony Optimization – to the Traveling Salesperson Problem, using the large-scale att48 instance as a test case. All three methods successfully produced high-quality tours much shorter than a naive initial route, underlining the value of metaheuristics for tackling NP-hard optimization problems. Among the algorithms, ACO achieved the best solution (closest to the known optimum), demonstrating the effectiveness of swarm intelligence and collective learning for TSP. The Genetic Algorithm was a close second, managing to find near-optimal solutions consistently and faster, highlighting the power of evolutionary search and good crossover design in recombining partial tours. Simulated Annealing, while not reaching the same level of optimality in the given time, still yielded solutions within a few percent of optimal and proved to be a reliable, straightforward approach.

Each algorithm has distinct strengths: SA is easy to implement and requires few resources, GA provides a good balance of exploration and exploitation via population dynamics, and ACO brings an intensification strategy that zooms in on elite solutions. The trade-offs observed include solution quality versus runtime (ACO gave the best result but took longer), and complexity versus performance (GA and ACO involve more complex operations than SA but achieved better outcomes). In practice, the choice of algorithm may depend on the specific requirements: if computation time and memory are limited, SA or GA might be preferred; if the absolute best solution is needed and resources allow, ACO (or a GA with heavy local search) would be justified.

Future Work: While our experiments provided valuable insights, there are several avenues for improvement and further exploration:

Hybrid Approaches: One promising direction is to combine the strengths of these algorithms. For example, a **Memetic Algorithm** could integrate SA's local search into GA: after each crossover, apply a few 2-opt or SA-like moves to the offspring. This hybrid could significantly improve GA's solution quality by ensuring each new tour is locally optimal. Similarly, one could use GA to diversify ACO: periodically use crossover between some of the best ant-found tours to create new starting tours or new pheromone trails. Hybrid **GA-ACO** systems or **GA-SA** systems have been reported to outperform their constituents. Exploring such hybrids on the att48 instance could yield even better results (potentially matching the exact optimum with less runtime). **Advanced Operators and Variants:** Within each algorithm, there is room to test advanced techniques. For GA, trying the **Edge Assembly Crossover (EAX)** – known for producing excellent TSP offspring – could improve performance. Incorporating **adaptive mutation rates** or **elitist archive** strategies might also help GA. For SA, experimenting with **different neighborhood moves** (like 3-opt moves or a mixture of move types) and **adaptive cooling** (where the schedule adapts if progress stalls) could result in faster convergence. For ACO, one could implement **Max-Min Ant System (MMAS)**variant which often avoids stagnation by limiting pheromone extremes, or use **multiple ant colonies** that occasionally exchange information, or apply **pheromone re-initialization** when the algorithm stagnates. Another extension for ACO is to parallelize the ant evaluations, which would drastically cut wall-clock time given enough processors.

Parameter Tuning and Automation: We manually tuned parameters for our experiments. In future work, one could use **automated parameter tuning** or **self-adaptive** mecha-

nisms so the algorithms adjust parameters on the fly (for instance, dynamically changing the cooling rate in SA or mutation rate in GA based on progress, or using an adaptive , in ACO). This would make the algorithms more robust to different instances without manual intervention.

Larger Instances and Generalization: It would be insightful to test these algorithms on even larger TSP instances (if available, e.g. TSPLIB's pla33810 with 33,810 nodes or pla85900 with 85,900 nodes) to see if the observations hold and how scalability is affected. Also, trying instances with different structures (random distances vs. Euclidean, clustered vs. uniform city distribution) could reveal how each algorithm copes with different landscape features. Our instance was Euclidean and planar, which favors 2-opt and ACO's distance heuristic; on a more arbitrary distance matrix, performance rankings might differ.

Machine Learning Integration: A modern line of research is using machine learning to aid classical algorithms. For TSP, **reinforcement learning (RL)** has been used to train policies that construct tours or improve existing ones. For example, one could train an RL agent to guide a local search (choosing which 2-opt move to try), or to select crossover points in GA, or to adjust pheromone evaporation rates in ACO dynamically. Another approach is using **graph neural networks** to predict good edges to include in a tour. These learned insights could be integrated as a module into SA/GA/ACO. For instance, a neural network could prioritize certain swaps in SA that are likely beneficial, effectively biasing the move selection with learned knowledge. Combining learning-based approaches with metaheuristics (sometimes called **"learned heuristics"**) is a promising area that could yield algorithms that get better with experience on classes of TSP instances.

Parallel and Distributed Computing: Given the large scale of the instance, leveraging parallelism is a logical next step. GA is naturally parallelizable (evaluate fitness of individuals in parallel, or run multiple sub-populations). ACO can run ants in parallel and then synchronize pheromone updates. SA can be parallelized by launching multiple independent runs (or a parallel tempering approach where multiple "temperatures" are run in parallel and occasionally swapped). Utilizing a multi-core or distributed setup could allow, for example, running 10 SA instances and taking the best, or running ACO with thousands of ants truly concurrently, thus cutting down time or enabling more iterations for even better results.

In summary, this project demonstrated the viability of SA, GA, and ACO for solving a real-world sized TSP and highlighted their comparative performance. The key findings are that metaheuristics can produce near-optimal solutions for a 48-city problem in reasonable time, with ACO slightly leading in quality and GA offering a very good quality-to-time trade-off. Future enhancements, especially hybrid algorithms and ML-assisted strategies, hold the potential to further improve solution quality or efficiency. The continual development of such approaches is important, as it not only advances TSP solving capabilities but also contributes broadly to tackling other complex combinatorial optimization problems with AI-inspired techniques.

References

[1] *The traveling salesman problem: a computational study*. Princeton series in applied mathematics. Princeton university press, Princeton (N.J.), 2006.

[2] Shyi-Ming Chen and Chih-Yao Chien. Solving the traveling salesman problem based on the genetic simulated annealing ant colony system with particle swarm optimization techniques. *Expert Systems with Applications*, 38(12):14439–14450, November 2011.

[3] Yanlan Deng, Juxia Xiong, and Qiuhong Wang. A Hybrid Cellular Genetic Algorithm for the Traveling Salesman Problem. *Mathematical Problems in Engineering*, 2021:1–16, February 2021.

[4] M. Dorigo and L.M. Gambardella. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, April 1997.

[5] M. Dorigo and L.M. Gambardella. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, April 1997.

[6] Michael R. Garey and David S. Johnson. *Computers and intractability: a guide to the theory of NP-completeness*. A series of books in the mathematical sciences. Freeman, New York [u.a], 27. print edition, 2009.

[7] David E. Goldberg and David E. Goldberg. *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley, Boston, 30. print edition, 2012.

[8] Keld Helsgaun. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, October 2000.

[9] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by Simulated Annealing. *Science*, 220(4598):671–680, May 1983.

[10] Gerhard Reinelt. TSPLIB—A Traveling Salesman Problem Library. *ORSA Journal on Computing*, 3(4):376–384, November 1991.

[11] Jiongzhi Zheng, Kun He, Jianrong Zhou, Yan Jin, and Chu-Min Li. Combining Reinforcement Learning with Lin-Kernighan-Helsgaun Algorithm for the Traveling Salesman Problem. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12445–12452, May 2021.